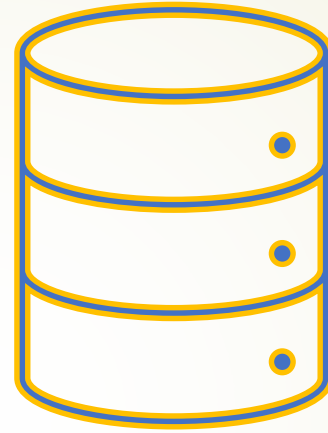




SQL: Data Definition and Manipulation Language

Federica Proietto Salantri

Definizione

SQL è il linguaggio di riferimento per le basi di dati relazionali;

Il nome originariamente "Structured Query Language", ora "nome proprio";

Sviluppato presso il laboratorio di ricerca IBM di S. Josè metà anni 70

Linguaggio con varie funzionalità:

- contiene sia il DDL (*Data Definition Language* - insieme di comandi per la definizione dello schema di una base di dati relazionale) sia il DML (*Data Manipulation Language* - insieme di comandi per la modifica e l'interrogazione dell'istanza di una base di dati)

SQL e standard

<i>Nome Informale</i>	<i>Nome Ufficiale</i>	<i>Caratteristiche</i>
SQL base	SQL - 86 SQL -89	Costrutti base Integrità referenziale
SQL-2	SQL-92	Modello relazionale 3 livelli: entry, intermediate, full
SQL-3	SQL:1999 SQL:2003 SQL:2016	Modello relazionale ad oggetti SQL/XML, Estensione Modello oggetti JSON, polymorphic table functions



Formalismo

<> Isolare un termine della sintassi

[] Il termine all'interno è opzionale

{ } Il termine all'interno può non comparire o comparire più volte

| deve essere scelto uno tra i termini separati dalle barre

Definizione dei dati in SQL

► Domini elementari:

- Caratteri: Il dominio `character` permette di rappresentare singoli caratteri o stringhe
`character [varying] [(lunghezza)]`
`[character set NomeFamigliaCaratteri]`

Es. Stringa di caratteri dell'alfabeto greco con lunghezza variabile max 1000:

```
character varying(1000) character set Greek
```

Si posso utilizzare anche le forme contratte `char` e `varchar`

Definizione dei dati in SQL

Domini elementari:

- Tipi numerici esatti: questa famiglia contiene i domini che permettono di rappresentare valori esatti o con una parte di lunghezza prefissata.
 - `numeric [(Precisione[, Scala])]`
 - `decimal [(Precisione[, Scala])]`
 - `integer`
 - `smallint`

`numeric(6,4) -99,9999 a 99,9999`

Definizione dei dati in SQL

➤ *Domini elementari:*

Tipi numerici approssimati: numeri approssimati tramite rappresentazione in virgola mobile (numero dato dalla coppia mantissa più esponente)

➤ float

➤ real

➤ double

Definizione dei dati in SQL

► Domini elementari:

Instanti temporali: Introdotti in SQL-2 per descrivere informazioni temporali.

► `date`

► `time [ (Precisione) ] [with time zone]`

► `timestamp [ (Precisione) ] [with time zone]`

Ciascuno di questi domini è strutturato e decomponibile in un insieme di campi. Il dominio *date* ammette i campi *year*, *month* e *day*. Il dominio *time* i campi *hour*, *minute*, *second*. Mentre *timestamp* tutti i campi da *year* a *second*.

Definizione dei dati in SQL

► Domini elementari:

- `interval PrimaUnitàDiTempo [to UltimaUnitàDiTempo]`
- `Interval year(5) to month` definisce intervalli fino a 99.999 anni e 11 mesi, mentre `interval day(4) to second(6)` definisce intervalli fino a 9.999 giorni, 23 ore, 59 minuti e 59,999999 secondi con una precisione di un milionesimo di secondo

Definizione dei dati in SQL

► Domini elementari: Introdotti in SQL-3

- Boolean: permette di rappresentare i singoli valori booleani (`true` e `false`);
- blob, clob (binary/character large object): per grandi immagini e testi. Per entrambi i domini il sistema garantisce di memorizzare solo il valore, ma non permette che esso venga utilizzato come criterio di selezione.

Definizione delle tabelle

- ▶ Una tabella SQL è costituita da una collezione ordinata di attributi e da un insieme di vincoli
- ▶ Istruzione `create table`:
 - ▶ definisce uno schema di relazione e ne crea un'istanza vuota
 - ▶ specifica attributi, domini e vincoli
- ▶ Sintassi:

```
create table NomeTabella  
(  
    NomeAttributo Dominio [ValoreDiDefault] [Vincoli]  
    {, (NomeAttributo Dominio [ValoreDiDefault] [Vincoli] }  
    AltriVincoli  
)
```



Definizione delle tabelle

```
CREATE TABLE Impiegato(  
    Matricola CHAR(6) PRIMARY KEY,  
    Nome      VARCHAR(20) NOT NULL,  
    Cognome   VARCHAR(20) NOT NULL,  
    Dipart    VARCHAR(15),  
    Stipendio NUMERIC(9) DEFAULT 0,  
  
    FOREIGN KEY(Dipart) REFERENCES  
        Dipartimento(NomeDip),  
    UNIQUE (Cognome, Nome)  
)
```

Definizione dei domini

Domini elementari (predefiniti)

Domini definiti dall'utente (semplici, ma riutilizzabili)

Sintassi:

- `create domain NomeDominio as TipoDiDato
[ValoreDiDefault] [Vincolo]`
- `create domain Voto as smallint DEFAULT NULL
check (value >=18 AND value <= 30)`

Vincoli Intrarelazionali



Vincoli: proprietà che devono essere verificate ad ogni istanza della base

Intrarelazionali;
Interrelazionali.



Il costrutto più potente per specificare vincoli generici è il check, il quale richiede di formulare delle interrogazioni sulle basi di dati.



I più semplici vincoli di tipo intrarelazionale sono not null, unique e primary key

Vincoli Intrarelazionali

NOT NULL: Il valore null non è ammesso come valore dell'attributo;

UNIQUE definisce chiavi:

```
Matricola character(6) unique;  
Nome varchar(20) not null,  
Cognome varchar(20) not null,  
unique (Nome, Cognome)
```

PRIMARY KEY: chiave primaria (una sola, implica NOT NULL):

```
Nome varchar(20) ,  
Cognome varchar(20) ,  
primary key (Nome, Cognome)
```


Vincoli Intrarelazionali

```
CREATE TABLE Impiegato(  
    Matricola CHAR(6) PRIMARY KEY,  
    Nome VARCHAR(20) NOT NULL,  
    Cognome VARCHAR(20) NOT NULL,  
    Dipart VARCHAR(15),  
    Stipendio NUMERIC(9) DEFAULT 0,  
  
    FOREIGN KEY(Dipart) REFERENCES  
        Dipartimento(NomeDip),  
    UNIQUE (Cognome, Nome)  
)
```

Vincoli Intrarelazionali

Nome CHAR(20) NOT NULL,

Cognome CHAR(20) NOT NULL,

UNIQUE (Cognome, Nome),

Nome CHAR(20) NOT NULL UNIQUE,

Cognome CHAR(20) NOT NULL UNIQUE,

Non è la stessa cosa!

Vincoli Interrelazionali

- ▶ I vincoli interrelazionali più diffusi e significativi sono i vincoli di integrità referenziale;
- ▶ Per la loro definizione si usa il **foreign key**, ovvero la chiave esterna
- ▶ Questo vincolo crea un legame tra i valori di un attributo della tabella su cui è definito (chiave interna) ed i valori di un attributo di un'altra tabella (chiave esterna).
- ▶ Unico vincolo: attributo tabella esterna sia definito come **unique**

Vincoli Interrelazionali

- ▶ Il valore del vincolo può essere definito in due modi, come `unique` e come `primary key`.
- ▶ Se c'è un solo attributo coinvolto, si può usare `references`, con il quale si specificano la tabella esterna e l'attributo della tabella esterna al quale l'attributo in questione deve essere legato.
- ▶ Quando invece il legame è rappresentato da un insieme di attributi si usa il `foreign key`.

Vincoli Interrelazionali

```
CREATE TABLE Impiegato(  
    Matricola CHAR(6) PRIMARY KEY,  
    Nome        VARCHAR(20) NOT NULL,  
    Cognome     VARCHAR(20) NOT NULL,  
    Dipart      VARCHAR(15)  
        REFERENCES DIPARTIMENTO(NOMEDIP),  
    Stipendio  NUMERIC(9) DEFAULT 0,  
    UNIQUE (Cognome, Nome)  
)
```

Vincoli Interrelazionali

- ▶ Se si volesse imporre che gli attributi Nome e Cognome debbano comparire in una tabella anagrafica allora si potrebbe aggiungere il vincolo:

Foreign key (Nome, Cognome)

References Anagrafica (Nome, Cognome)

- ▶ Nome e Cognome impiegato corrispondono a nome e cognome anagrafica



Vincoli Interrelazionali

- SQL permette di scegliere delle azioni da adottare quando viene rilevata una violazione di integrità referenziale.
- Le operazioni sulla tabella esterna che possono introdurre delle violazioni sono le modifiche dell'attributo riferito e la cancellazione di righe (nel ns es. cancellazioni di righe di DIPARTIMENTO e le modifiche dell'attributo NOME)

Vincoli Interrelazionali

- ▶ In caso di vincoli di integrità referenziale, quando la violazione avviene per un cambiamento apportato alla tabella esterna, si deve associare una politica alle operazioni che possono causare inconsistenza:

on < delete | update >

<cascade|set null|set default|no action>

subito dopo la specifica del riferimento (references)

Vincoli Interrelazionali

Modifica (comando **update**):

cascade il nuovo valore dell'attributo della tabella esterna viene riportato su tutte le corrispondenti righe della tabella interna.

set null all'attributo referente viene assegnato il valore nullo.

set default all'attributo referente viene assegnato il valore di default.

no action la modifica non viene consentita



Vincoli Interrelazionali

Cancellazione (comando **delete**):

cascade tutte le corrispondenti righe della tabella interna vengono cancellate.

set null all'attributo referente viene assegnato il valore nullo.

set default all'attributo referente viene assegnato il valore di default.

no action la cancellazione non viene consentita.

Vincoli Interrelazionali

```
CREATE TABLE Impiegato(  
    Matricola CHAR(6) PRIMARY KEY,  
    Nome      CHAR(20) NOT NULL,  
    Cognome   CHAR(20) NOT NULL,  
    Dipart    VARCHAR(15)  
        REFERENCES DIPARTIMENTO(NOMEDIP)  
        on delete set null  
        on update cascade,  
    Stipendio FLOAT(9) DEFAULT 0,  
    UNIQUE (Cognome, Nome)  
)
```

Esempio

Infrazioni

<u>Codice</u>	Data	Vigile	Prov	Numero
34321	1/2/95	3987	MI	39548K
53524	4/3/95	3295	TO	E39548
64521	5/4/96	3295	PR	839548
73321	5/2/98	9345	PR	839548

Vigili

Matricola	Cognome	Nome
3987	Rossi	Luca
3295	Neri	Piero
9345	Neri	Mario
7543	Mori	Gino

Create Table - Esempio

```
CREATE TABLE Infrazioni (  
    Codice          CHAR(6) NOT NULL PRIMARY KEY,  
    Data            DATE NOT NULL,  
    Vigile           INTEGER NOT NULL  
                    REFERENCES Vigili(Matricola),  
    Provincia        CHAR(2),  
    Numero           CHAR(6)  
);
```

Esempio

Infrazioni

<u>Codice</u>	Data	Vigile	Prov	Numero
34321	1/2/95	3987	MI	39548K
53524	4/3/95	3295	TO	E39548
64521	5/4/96	3295	PR	839548
73321	5/2/98	9345	PR	839548

Auto

<u>Prov</u>	<u>Numero</u>	Cognome	Nome
MI	39548K	Rossi	Mario
TO	E39548	Rossi	Mario
PR	839548	Neri	Luca

Create Table - Esempio

```
CREATE TABLE Infrazioni(  
    Codice          CHAR(6) PRIMARY KEY,  
    Data            DATE NOT NULL,  
    Vigile          INTEGER NOT NULL  
                    REFERENCES Vigili(Matricola),  
    Provincia       CHAR(2),  
    Numero          CHAR(6) ,  
  
    FOREIGN KEY(Provincia, Numero)  
        REFERENCES Auto(Provincia, Numero)  
)
```



Modifica degli schemi

- ▶ E' possibile modificare gli schemi con i comandi `alter` e `drop`.
- ▶ **alter** consente di modificare domini e schemi di tabelle.

Modifica degli schemi

```
alter table NomeTabella
```

```
<
```

```
    alter column NomeAttributo
```

```
        <set default NuovoDefault | drop default > |
```

```
    add constraint DefVincolo |
```

```
    drop constraint NomeVincolo |
```

```
    add column NomeAttributo |
```

```
    drop column NomeAttributo
```

```
>
```



Modifica degli schemi

```
alter table Dipartimento add column NumUfficio  
numeric(4)
```

```
alter table Persone alter column DataNascita  
year
```

```
alter table Ordini add FOREIGN KEY (PersonID)  
REFERENCES Persone(PersonID)
```

Modifica degli schemi

Il comando **drop** permette di rimuovere dei componenti

```
drop < domain | table | view | assertion >  
    NomeElemento  
    [ restrict | cascade ]
```

- **restrict** specifica di non eseguire il comando in presenza di oggetti non vuoti.
- **cascade** implica che gli oggetti specificati siano rimossi. Inoltre vengono rimossi tutti gli oggetti da essi dipendenti.



Modifica degli schemi

Esempio:

```
drop domain StringaLunga cascade
```

Dove *StringaLunga* è definita come `varchar(100)`.

Quando si rimuove una tabella con l'opzione `cascade` tutte le righe vengono perse, se la tabella compare in qualche definizione di qualche altra tabella allora anche queste vengono rimosse



Interrogazione

- ▶ La parte di SQL dedicata alla formulazione di interrogazioni fa parte del DML
 - ▶ L'interrogazione viene passata *all'ottimizzatore di interrogazioni (query optimizer)* che fa parte del DBMS. Questo la analizza e la traduce nel linguaggio di interrogazione interno al DBMS.
 - ▶ Per questo chi programma in SQL deve cercare di scrivere codice *leggibile e facilmente modificabile* oltre che efficiente.
- 

Interrogazione

- ▶ Le operazioni di interrogazione vengono specificate per mezzo dell'istruzione `select`.

```
select ListaAttributi      (target list)
from ListaTabelle         (clausola from)
[where Condizione ]       (clausola where)
```

Più in dettaglio:

```
select AttrEspr [[as] Alias]{,AttrEspr [[as]
Alias]}

from Tabella [[as] Alias]{,Tabella [[as]
Alias]}
[ where Condizione ]
```

Istruzione SQL seleziona, tra le righe che appartengono al prodotto cartesiano delle tabelle indicate nella clausola *from*, quelle che soddisfano le condizioni espresse nell'argomento della clausola *where*

Interrogazione

Es. Data una base di dati che contiene le tabelle:

```
IMPIEGATO  (Nome, Cognome, Dipart, Ufficio,  
           Stipendio, Città)  
DIPARTIMENTO (Nome, Indirizzo, Città)
```

```
select Stipendio/12 as SalarioMensile  
from    Impiegato  
where   Cognome = `Rossi`
```

Il risultato è una tabella con una colonna rinominata *SalarioMensile* e tante righe quanti sono gli impiegati che si chiamano Rossi.

Se si usa ***** dopo **select** si selezionano tutti gli attributi

Esempio Interrogazione

Nome	Cognome	Dipart.	Ufficio	Stipendio	Città
Mario	Rossi	Ammin.	10	45	Milano
Carlo	Bianchi	Prod.	20	36	Torino
Giovanni	Verdi	Ammin.	20	40	Roma
Franco	Neri	Distrib.	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Paola	Rosati	Ammin.	75	46	Venezia

```
select Stipendio/12 as SalarioMensile
from   Impiegato
where  Cognome = `Rossi`
```

SalarioMensile
3,75
6,66

Esempio Interrogazione

Nome	Cognome	Dipart.	Ufficio	Stipendio	Città
Mario	Rossi	Ammin.	10	45	Milano
Carlo	Bianchi	Prod.	20	36	Torino
Giovanni	Verdi	Ammin.	20	40	Roma
Franco	Neri	Distrib.	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Paola	Rosati	Ammin.	75	46	Venezia

```
select *from   Impiegato where   Cognome = `Rossi`
```

Nome	Cognome	Dipart.	Ufficio	Stipendio	Città
Mario	Rossi	Ammin.	10	45	Milano
Carlo	Rossi	Direzione	14	80	Milano

Selezione e proiezione con SQL

Nome e reddito delle persone con meno di trenta anni

$\text{PROJ}_{\text{Nome, Reddito}}(\text{SEL}_{\text{Eta} < 30}(\text{Persone}))$

Proiezione

```
select nome, reddito
from persone
where eta < 30
```

Selezione

```
select p.nome as nome, p.reddito as reddito
from persone as p
where p.eta < 30
```

Interrogazioni su più tabelle

- ▶ Se si vogliono estrarre informazioni da più tabelle, tutte le tabelle devono apparire come argomento della clausola **from**.
- ▶ Se si deve formulare un join, è possibile farlo in modo esplicito (**from** reggerà allora un join fra le tabelle)
- ▶ Infatti, specificare una lista di tabelle nella clausola **from** equivale ad eseguire la query sul prodotto cartesiano delle tabelle riportate nella lista, e un theta-join equivale a un prodotto cartesiano seguito da una selezione sulla condizione di join.

Interrogazioni su più tabelle

Es. Estrarre i nomi degli impiegati e le città dove lavorano.

```
select Impiegato.Nome, Impiegato.Cognome,  
        Dipartimento.Città  
from    Impiegato, Dipartimento  
where Impiegato.Dipart=Dipartimento.Nome
```

- E' necessario specificare il nome della tabella quando le tabelle presenti nella clausola from posseggono più attributi con lo stesso nome

Interrogazioni su più tabelle

Es. Estrarre i nomi degli impiegati e le città dove lavorano.

```
Select I.Nome, I.Cognome, D.Città  
from    Impiegato as I, Dipartimento as D  
where   Dipart = D.Nome
```

E' necessario specificare il nome della tabella quando le tabelle presenti nella clausola from posseggono più attributi con lo stesso nome

Interrogazioni su più tabelle

```
Select I.Nome,I.Cognome,D.Città
from   Impiegato as I, Dipartimento as D
where  Dipart = D.Nome
```

IMPIEGATO

Nome	Cognome	Dipart.	Ufficio	Stipendio
Mario	Rossi	Ammin.	10	45
Carlo	Bianchi	Prod.	20	36
Giovanni	Verdi	Ammin.	20	40
Franco	Neri	Distrib.	16	45
Carlo	Rossi	Direzione	14	80
Paola	Rosati	Ammin.	75	46

DIPARTIMENTO

Nome	Indirizzo	Città
Ammin.	Via T, 2	Milano
Prod.	Via P,2	Torino
Distrib.	Via C, 3	Roma
Direzione	Via S, 5	Milano
Ricerca.	Via TS,89	Milano

Nome	Cognome	Città
Mario	Rossi	Milano
Carlo	Bianchi	Torino
Giovanni	Verdi	Roma
Franco	Neri	Napoli
Carlo	Rossi	Milano
Paola	Rosati	Venezia

Clausola Where

- ▶ Ammette come argomento una condizione logica.
- ▶ Gli operatori ammessi per i predicati semplici (confronto attributo-costante o attributo-espressione) sono =, <>, <, >, <=, >=
- ▶ I predicati semplici possono essere modificati tramite gli operatori logici **and**, **or**, **not**.
- ▶ **not** ha precedenza su **and** e **or**, ma non è definita la precedenza fra **and** e **or**. Quando si coordinano più predicati con **and** e **or** è bene esplicitare le precedenze con le parentesi.

Es.

```
select Nome  
from Impiegato  
where Cognome = 'Rossi' and (Dipart = 'Amministratz' or  
Dipart = 'Produz')
```

Esempio Interrogazione

Nome	Cognome	Dipart.	Ufficio	Stipendio	Città
Mario	Rossi	Ammin.	10	45	Milano
Carlo	Bianchi	Prod.	20	36	Torino
Giovanni	Verdi	Ammin.	20	40	Roma
Franco	Neri	Distrib.	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Paola	Rosati	Ammin.	75	46	Venezia

```
select Nome
from Impiegato
where Cognome = 'Rossi' and (Dipart = 'Ammin.' or
Dipart = 'Prod.')
```

Nome
Mario

Operatore Like

Per i confronti fra stringhe è definito anche l'operatore **like**. Il confronto è effettuato con una stringa che può contenere i caratteri speciali % e _ .

- _ rappresenta un carattere arbitrario
- % rappresenta un numero arbitrario di caratteri (anche zero).

Es.

```
select *  
from Impiegato  
where Cognome like '_o%i'
```

La condizione è soddisfatta da Rossi, Poli, Pollastri, ecc.

Operatore Like

- ▶ Le persone che hanno un nome che inizia per 'A' e ha una 'd' come terza lettera

```
select *  
from persone  
where nome like 'A_d%'
```

Gestione dei valori nulli

- ▶ La gestione dei valori nulli, a seconda dell'implementazione, avviene attraverso una logica a due valori come nell'SQL-89, o a tre valori (vero, falso, unknown) come nell'SQL-2.
- ▶ Le condizioni sui valori nulli possono essere definite attraverso i predicati **is null** o **is not null**.

Gestione dei valori nulli

Impiegati

Matricola	Cognome	Filiale	Età
5998	Neri	Milano	45
9553	Bruni	Milano	NULL

Gli impiegati la cui età è o potrebbe essere maggiore di 40

```
SEL Età > 40 OR Età IS NULL (Impiegati)
```

```
select *  
from impiegati  
where eta > 40 or eta is null
```

Duplicati

- L'algebra relazionale non ammette duplicati, SQL li ammette.
- Quindi

```
select Città from Persona  
      where Cognome= 'Rossi'
```

estrae una lista di città in cui una città può comparire più volte.

- Per evitare i duplicati SQL prevede la specifica **distinct** da inserire subito dopo **select**.

```
select distinct Città from Persona  
      where Cognome= 'Rossi'
```

Duplicati

Matricola	Cognome	Nome	Città
1	Rossi	Mario	Verona
2	Bianchi	Carlo	Roma
3	Rossi	Giovanni	Verona
4	Rossi	Pietro	Milano

```
select Città  
from Persona  
where Cognome= 'Rossi'
```

Città
Verona
Verona
Milano

Duplicati

Matricola	Cognome	Nome	Città
1	Rossi	Mario	Verona
2	Bianchi	Carlo	Roma
3	Rossi	Giovanni	Verona
4	Rossi	Pietro	Milano

```
select distinct Città  
from Persona  
where Cognome= 'Rossi'
```

Città
Verona
Milano

Join

- ▶ In SQL-2 è stata introdotta la seguente sintassi per esprimere il join ed estenderlo ai join esterni

```
select AttrEspr [[as] Alias]{,AttrEspr [[as] Alias]}  
from Tabella [[as] Alias]  
{[TipoJoin] join Tabella [[as]Alias] on CondizioneJoin}  
[ where AltraCondizione ]
```

- ▶ *TipoJoin* può assumere i valori
 - ▶ **inner**, **right**[**outer**], **left**[**outer**], **full**[**outer**]
 - ▶ **inner** è il default.
- ▶ C'è anche l'estensione **natural** che implica la condizione di uguaglianza sugli attributi con lo stesso nome.

Join

Maternità

Madre	Figlio
Luisa	Maria
Luisa	Luigi
Anna	Olga
Anna	Filippo
Maria	Andrea
Maria	Aldo

Paternità

Padre	Figlio
Sergio	Franco
Luigi	Olga
Luigi	Filippo
Franco	Andrea
Franco	Aldo

Persone

Nome	Età	Reddito
Andrea	27	21
Aldo	25	15
Maria	55	42
Anna	50	35
Filippo	26	30
Luigi	50	40
Franco	60	20
Olga	30	41
Sergio	85	35
Luisa	75	87

Join implicito ed esplicito

Padre e madre di ogni persona

```
select paternita.figlio, padre, madre  
from    maternita, paternita  
where   paternita.figlio = maternita.figlio
```

```
select paternita.figlio, padre, madre  
from    maternita join paternita on  
         paternita.figlio = maternita.figlio
```


Ordinamento

- ▶ E' possibile anche ordinare le righe del risultato di una interrogazione attraverso la clausola **order by**, a chiusura di una interrogazione.

```
order by AttrdiOrdinamento [asc | desc]  
        {, AttrdiOrdinamento [asc | desc] }
```

- ▶ **asc** (default) indica ordinamento ascendente, **desc** discendente.
- ▶ Il primo attributo ha priorità, a parità di valore il secondo ecc.

```
select *  
from Persona  
order by Cognome, Nome
```



SELECT

La forma di select cui siamo arrivati dopo le estensioni viste è quindi:

```
select ListaAttributiOEspressioni  
from ListaTabelle (o Join)  
[ where CondizioniSemplici ]  
[ order by ListaAttributiDiOrdinamento ]
```



Manipolare dati in SQL

- ▶ I comandi che permettono di modificare il contenuto delle basi di dati sono:
 - ▶ insert
 - ▶ delete
 - ▶ update
- ▶ Condizione che può coinvolgere anche altre relazioni

Inserimento

```
INSERT INTO Tabella [ ( Attributi ) ]  
    <VALUES ( Valori ) | select SQL>
```

Esempio:

1) **Insert into** Dipartimento (NomeDip, Città)
 values ('Produzione', 'Torino');

2) **insert into** ProdottiMilanesi
 (**select** Codice, Descrizione
 from Prodotto
 where LuogoProd = 'Milano')



INSERIMENTO

```
INSERT INTO Persone VALUES ('Mario', 25, 52)
```

```
INSERT INTO Persone (Nome, Eta, Reddito)  
VALUES ('Pino', 25, 52)
```

```
INSERT INTO Persone (Nome, Reddito)  
VALUES ('Lino', 55)
```

```
INSERT INTO Persone (Nome)  
SELECT Padre  
FROM Paternita  
WHERE Padre NOT IN (SELECT Nome FROM Persone)
```



Inserimento

- ▶ l'ordinamento degli attributi (se presente) e dei valori è significativo
- ▶ le due liste debbono avere lo stesso numero di elementi
- ▶ se la lista di attributi è omessa, si fa riferimento a tutti gli attributi della relazione, secondo l'ordine con cui sono stati definiti
- ▶ se la lista di attributi non contiene tutti gli attributi della relazione, per gli altri viene inserito un valore nullo (che deve essere permesso) o un valore di default

Cancellazione

```
DELETE FROM Tabella  
[ WHERE Condizione ]
```

- ▶ Quando la condizione non viene specificata, il comando cancella tutte le righe altrimenti solo le righe che soddisfano la condizione
- ▶ Es.

```
delete from Dipartimento  
where Nome = 'Prod.'
```




Cancellazione

- ▶ `delete from Dipartimento`

- ▶ `drop table Dipartimento cascade`



Eliminazione

- ▶ Elimina le ennuple che soddisfano la condizione;
 - ▶ Può causare (se i vincoli di integrità referenziale sono definiti con politiche di reazione cascade) eliminazioni da altre relazioni
 - ▶ Se la where è omessa si intende where true
- 

Modifica

```
UPDATE NomeTabella  
SET  Attributo = < Espressione |  
                                SELECT ... |  
                                NULL |  
                                DEFAULT >  
[ WHERE Condizione ]
```

- Aggiornare uno o più attributi delle righe di *NomeTabella* che soddisfano la *Condizione*.



Modifica

Update Impiegato

```
set Stipendio = StipendioBase + 5  
where Matricola = 'M2047';
```

Update Impiegato

```
set Stipendio = Stipendio * 1.1  
where Dipart = 'Ammin.';
```